



Obligatorio - Bases de Datos I

Juan Ignacio Weyrauch
María Belén Mendes
Matías Lluis

Universidad Católica del Uruguay
Docente Fernando Scoccozza
15 de Junio de 2026

Introducción

En este informe se presentan las decisiones de diseño tomadas para desarrollar el sistema de gestión de actividades deportivas universitarias. El propósito del sistema es reemplazar la gestión manual realizada mediante planillas, facilitando el manejo de las inscripciones, el control de los cupos y el registro de las asistencias de una forma más organizada y eficiente. Para su implementación se diseñó una base de datos en MySQL utilizando el motor InnoDB, aplicando principios de normalización y distintas restricciones de integridad tanto en el esquema de la base de datos como en la lógica del backend.

Modelo de Datos

El modelo relacional está compuesto por ocho tablas que representan las entidades principales del sistema y sus relaciones.

Entidades Identificadas

Entidad	Descripción	Clave Primaria
facultad	Facultad a la que pertenece una carrera	id_facultad (AUTO_INCREMENT)
carrera	Carrera universitaria. Depende de facultad.	id_carrera (AUTO_INCREMENT)
estudiante	Estudiante registrado en el sistema	id_estudiante (AUTO_INCREMENT)
disciplina	Tipo de deporte (fútbol, yoga, etc.)	id_disciplina (AUTO_INCREMENT)
espacio	Lugar físico donde se realiza una actividad	id_espacio (AUTO_INCREMENT)
actividad	Actividad deportiva concreta	id_actividad (AUTO_INCREMENT)
inscripcion	Entidad asociativa que resuelve la relación M:N entre estudiante y actividad	id_inscripcion (AUTO_INCREMENT)
asistencia	Un único registro de asistencia por inscripción para cada fecha.	id_asistencia (AUTO_INCREMENT)

Decisiones Tomadas

Separación de facultad y carrera

Se decidió modelar facultad y carrera como tablas independientes en lugar de almacenarlas como atributos del estudiante. Esta decisión evita redundancia de datos y permite mantener un catálogo controlado de carreras y facultades. Un estudiante referencia su carrera mediante clave foránea, y la carrera a su vez referencia la facultad. Esto permite consultar fácilmente la cantidad de inscriptos por carrera o facultad sin depender de texto libre.

Claves candidatas con UNIQUE

Se identificaron algunos atributos que debían ser únicos además de la clave primaria, por lo que se establecieron restricciones UNIQUE para garantizar su integridad:

- **estudiante.documento**: evita que existan dos estudiantes con el mismo documento.
- **estudiante.email**: asegura que cada estudiante tenga un correo electrónico único.
- **facultad.nombre y disciplina.nombre**: impiden la existencia de registros duplicados en estos catálogos.
- **espacio (nombre, ubicacion)**: la combinación de ambos campos debe ser única, lo que permite que existan espacios con el mismo nombre siempre que se encuentren en ubicaciones diferentes.
- **inscripcion (id_estudiante, id_actividad)**: garantiza que un estudiante no pueda inscribirse más de una vez en la misma actividad.
- **asistencia (id_inscripcion, fecha)**: asegura que exista un único registro de asistencia por inscripción para cada fecha.
- **carrera.nombre UNIQUE(nombre)**: no pueden haber dos carreras con el mismo nombre.

Uso de ENUM para valores controlados

Los atributos que solo pueden tomar valores de un conjunto fijo se declararon como ENUM en lugar de VARCHAR. Esto garantiza que la base de datos rechace cualquier valor fuera del conjunto permitido, sin depender de una validación externa.

Campo	Tabla	Valores Posibles
estado	actividad	abierta, cerrada, finalizada, cancelada
dia_semana	actividad	lunes, martes, miercoles, jueves, viernes, sabado, domingo
estado	inscripcion	confirmada, en_espera, cancelada

Restricciones con Check

También se utilizaron restricciones CHECK para asegurar que ciertos valores numéricos sean válidos directamente desde la base de datos. En actividad.cupo_maximo se estableció que el valor debe ser mayor que cero, garantizando que todas las actividades tengan un cupo positivo. Por su parte, en inscripcion.posicion_espera se definió que este campo puede ser nulo cuando el estudiante no está en lista de espera, pero si tiene un valor asignado, este debe ser mayor que cero.

Implementación de validaciones del sistema

El sistema valida los datos en tres capas, cada una con un propósito distinto y sin depender de las demás:

- Frontend: controla el formato y previene acciones inválidas antes de que lleguen al backend. Se hace a través de los formularios, verificando campos obligatorios y tipos de datos ingresados, así como también los desplegables muestran solamente actividades en estado “abierta”, y el de asistencia solo inscripciones “confirmada”, logrando que el usuario no pueda violar reglas de negocio.
- Backend: concentra las reglas de negocio, antes de cada operación se revalida en el servidor (actividad abierta, cupo no superado, etc.).
- Base de datos: garantiza los invariantes mediante restricciones de diseño (UNIQUE, CHECK, ENUM y claves foráneas).

Validación	Implementación
Solo permitir inscripciones en actividades abiertas	El backend consulta el estado de la actividad antes de insertar la inscripción.
No superar el cupo máximo	Se cuenta la cantidad de inscripciones confirmadas y se compara con cupo_maximo.
Enviar a lista de espera cuando no hay cupo	Si las inscripciones confirmadas alcanzan el cupo_maximo, la nueva inscripción se registra con estado en_espera y se asigna una posicion_espera.
Evitar inscripciones duplicadas	La restricción UNIQUE (id_estudiante, id_actividad) en la tabla inscripcion impide que un estudiante se inscriba dos veces en la misma actividad.
Registrar asistencia solo a estudiantes confirmados	Antes de insertar una asistencia, el backend verifica que la inscripción tenga estado confirmada.

Índices Adicionales

Además de los índices que se crean automáticamente a partir de las claves foráneas, se agregaron algunos índices adicionales para mejorar el rendimiento de las consultas utilizadas en los reportes del sistema. En particular, ``idx_actividad_estado`` permite filtrar más rápidamente las actividades según su estado, ``idx_inscripcion_actividad_estado`` optimiza el conteo de inscripciones confirmadas para cada actividad y ``idx_asistencia_presente`` agiliza las consultas relacionadas con el cálculo del porcentaje de asistencia.

Consultas Implementadas

Se implementaron las siete consultas requeridas por la letra, más las tres consultas adicionales propuestas por nuestro equipo:

- Actividades con mayor cantidad de inscriptos confirmados
- Actividades con cupos disponibles
- Cantidad de inscriptos por disciplina deportiva
- Cantidad de inscriptos por carrera y facultad
- Porcentaje de ocupación de cada actividad
- Porcentaje de asistencia por actividad
- Estudiantes con tres o más inasistencias registradas
- (Propia) Lista de espera por actividad.
- (Propia) Uso de espacios deportivos.
- (Propia) Estudiantes más activos.

Conclusión

El diseño de la base de datos cumple con los requerimientos del enunciado, aplicando normalización, restricciones de integridad referencial y validaciones tanto en el esquema como en el backend. La separación en entidades bien definidas permite escalar el sistema fácilmente en el futuro, por ejemplo agregando nuevos tipos de disciplinas o espacios sin modificar la estructura existente.